task2

September 11, 2026

[57]:
```
import pandas as pd

data = pd.read_csv("heart-disease.csv")
data_shuffled = data.sample(frac=1, random_state=42).reset_index(drop=True)

features = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",␣
 ↪"thalach", "exang", "oldpeak", "slope", "ca", "thal"]
X = data_shuffled[features]
y = data_shuffled["target"]

print(X.head())
print(y.head())
```

```
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   57    1   0       150   276    0        0      112      1      0.6      1
1   59    1   3       170   288    0        0      159      0      0.2      1
2   57    1   2       150   126    1        1      173      0      0.2      2
3   56    0   0       134   409    0        0      150      1      1.9      1
4   71    0   2       110   265    1        0      130      0      0.0      2

   ca  thal
0   1     1
1   0     3
2   1     3
3   2     3
4   1     2
0    0
1    0
2    1
3    0
4    1
Name: target, dtype: int64
```

[58]:
```
from math import e

def sigmoid(z):
    return 1 / (1 + e ** (-z))
```

```
def standard_scaler(X):
    for col in X.columns:
        mean = X[col].mean()
        std = X[col].std()
        X[col] = (X[col] - mean) / std

    return X
```

[59]:
```
import numpy as np

def gradient_descent(X, y, learning_rate, epochs=1000):
    m, n = X.shape
    w = np.zeros(n)

    for _ in range(epochs):
        z = np.dot(X, w)
        h = sigmoid(z)
        gradient = np.dot(X.T, (y - h)) / -m
        w = w - learning_rate * gradient

    return w

def newtons_method(X, y, epochs=1000):
    m, n = X.shape
    theta = np.zeros(n)

    for _ in range(epochs):
        z = np.dot(X, theta)
        h = sigmoid(z)
        gradient = np.dot(X.T, (y - h)) / -m
        S = np.diag((1 - h) * h)
        hessian = np.dot(X.T, np.dot(S, X)) / m
        try:
            theta = theta - np.dot(np.linalg.inv(hessian), gradient)
        except np.linalg.LinAlgError:
            print(f"Hessian is singular and not invertible.\nIteration no: {_}")
            break

    return theta
```

[60]:
```
X_scaled = standard_scaler(X)
X_scaled_b = np.c_[np.ones((X_scaled.shape[0], 1)), X_scaled]

w_gd = gradient_descent(X_scaled_b, y, learning_rate=0.1)
theta_newton = newtons_method(X_scaled_b, y)
```

[61]:
```
def predict(X, theta):
    probabilities = sigmoid(np.dot(X, theta))
    y_pred = []

    for p in probabilities:
        if p > 0.5:
            y_pred.append(1)
        else:
            y_pred.append(0)

    return y_pred

predict_gd = predict(X_scaled_b, w_gd)
predict_newton = predict(X_scaled_b, theta_newton)
```

[62]:
```
from sklearn.metrics import accuracy_score

acc_gd = accuracy_score(y, predict_gd)
acc_newton = accuracy_score(y, predict_newton)

print(f"Gradient Descent Accuracy: {acc_gd}")
print(f"Newton's Method Accuracy:  {acc_newton}")
```

```
Gradient Descent Accuracy: 0.8514851485148515
Newton's Method Accuracy:  0.8514851485148515
```

[63]:
```
results_df = pd.DataFrame({
    "Original Target": y,
    "predict(GD)": predict_gd,
    "predict(Newton)": predict_newton
})

print(results_df.head())
```

```
   Original Target  predict(GD)  predict(Newton)
0                0            0                0
1                0            1                1
2                1            1                1
3                0            0                0
4                1            1                1
```